INF201 - Deliverable 3
Declaration of Generative AI Use

Scope: Tasks 1-3 (Character, Dice and Weapon classes), including the Python modules and Jupyter Notebook demonstrations.

How AI was used

I used ChatGPT as an interactive learning and debugging assistant while working on Tasks 1-3. I worked in VS Code and Jupyter Notebook, tried to write parts of the code myself, ran tests, shared screenshots and error messages, and asked for explanations and corrections. ChatGPT also supplied example code and, especially for Task 3, complete code snippets that I adapted and entered into my files. This declaration therefore does not claim that every line was written without AI assistance.

Task 1 - Character

- I asked how to define take_damage(), subtract HP, and keep HP from falling below zero using max().
- I asked how to implement heal(), learned why min() prevents HP from exceeding hp_max, and corrected attribute-name mistakes.
- I attempted the is_alive implementation and asked whether it met the assignment requirement. ChatGPT pointed out that it must be a read-only @property and helped me remove a duplicate method.
- I asked how attack() should call the target's take_damage() instead of modifying its protected HP directly.
- I requested PEP 8 formatting, type hints and docstrings, and checked the complete class through screenshots.
- I tested damage, healing, HP limits, alive/dead status and attacks in Jupyter, reporting the observed results.

Task 2 - Dice

- I asked how to construct a dice with at least two sides, why the validation checks sides < 2, and how to store self.sides.
- I wrote or attempted expressions for roll(), __repr__() and __call__(); ChatGPT explained and corrected the syntax and showed how __call__() reuses roll().
- I asked why importing Dice failed. With ChatGPT's guidance I saved the Python file, restarted the Jupyter kernel and reran the import.
- I tested six- and twenty-sided dice, direct object calls and rejection of invalid dice.

Task 3 - Weapon

- I shared the assignment requirements for items.py and asked to work faster. ChatGPT provided a full example Weapon class and suggested changes to Character.__init__() and attack().
- I asked where to document the optional weapon parameter and incorporated the explanation in the constructor docstring.
- I shared Jupyter errors about the weapon argument and an undefined Weapon name. ChatGPT helped me troubleshoot saving, restarting and running import cells in the correct order.
- I tested attacks with a weapon (random dice damage) and without a weapon (default damage of 10), and asked for concise English code comments.

Documentation and responsibility

ChatGPT also helped with the organization of Python modules versus the Jupyter Notebook, short discussion/conclusion text, and Git/GitHub instructions. I reviewed and ran the code and tests in my own environment. The submitted files and results should be assessed as my work with the AI assistance described above.

Tool: OpenAI ChatGPT. Use: explanations, code examples, debugging, documentation and workflow guidance.